Forward Deployed Engineering from First Principles to Enterprise Delivery — Intermediate

Portfolio projects · lessons 10–20

Each takes 1–3 days and ends with something to show an employer.

Project 1: Gate a triage prompt change with an eval set and three graders

Goal: Build a 60-case labelled eval set for the Basket Bazaar triage feature, score the shipped and candidate prompts against it, grade the replies three ways, and block a prompt change that breaks a shipped case, the release discipline hiring managers ask about first for any customer AI feature.

Uses lessons: 05, 10, 11, 12, 18, 20

1. Write data/eval_set.csv with the lesson 11 columns (case_id, order_id, message, want): the 12 cases from lesson 11 plus 48 new invented ones, 10 per label across missing_item, wrong_item, late, damaged and other, and write each want label by hand before the first model call.

2. Check every order_id against the clean orders file from lesson 05 with a script that exits with an error on any unknown id, and print the count of cases per label (it must read 10 for each of the five).

3. Run the shipped prompt on all 60 cases in one request (the lesson 11 pattern, through llm_setup.get_llm() under llm_replay) and save the verdicts to data/scored_shipped.json; then write a candidate prompt with two added rules (a count of items is missing_item, a broken item is damaged) and save its verdicts to data/scored_candidate.json.

4. Write tests/test_gate.py with pytest: it fails and prints the case id when any case that passed under the shipped prompt fails under the candidate, and when the candidate passes fewer than 54 of 60 cases (90 percent).

5. Draft one reply per case with the lesson 12 reply prompt in one request (60 replies), then grade the replies with 5 code rules (names the order id, no money amount, no time promise, 15 to 60 words, no guarantee or always), a model grader with a rubric of at most 10 lines, and a person's review of 20 random replies (random seed 7); compute percent agreement and Cohen's kappa (sklearn.metrics.cohen_kappa_score) of each automatic grader against the person.

6. Add a second gate test that fails when either automatic grader has kappa below 0.6 on the person's 20 replies, run pytest -q and save its exit code to data/gate_exit.txt.

7. Read the input and output tokens of each of the four model calls from the usage fields, price them at the lesson 18 list prices (USD 0.25 per million input tokens and 1.50 per million output tokens, dated 2026-10-22; print the date next to the cost), divide by 60 for cost per case, and write one results table in README.md.

Deliverable: A repo with data/eval_set.csv (60 rows), the scorer, the code rules, the model grader, the person's 20-reply sample, tests/test_gate.py and README.md. The README reports, from your own run: shipped and candidate pass counts out of 60, the regressed cases by id (the gate must name each one), percent agreement and kappa for the code and model graders against the person, input and output tokens per call, cost per case in USD to four decimals with the price date, and the pytest exit code.

Stretch: Add a second human reviewer on the same 20 replies, report kappa between the two people, and explain why the automatic gate cannot be stricter than that number. · Add a nightly script that scores 200 newly generated cases and fails when the pass rate drops by more than 5 points from the last release.

Project 2: Ship the triage service into a private network with scoped access and a tested rollback

Goal: Deploy the Basket Bazaar triage service with Docker Compose into a simulated customer network that has no internet route, with an egress allowlist, scoped service accounts and three runbooks, and prove each control with a measured result, the way a security review and an on-call handover ask you to.

Uses lessons: 13, 14, 15, 16, 17

1. Start the lab customer API from lesson 13 with its two service accounts, bb-orders-reader (scope orders:read) and bb-credit-bot (scope orders:read and credits:write), and confirm each token's lifetime is 300 seconds from the expires_in field of the token response.

2. Write a Dockerfile and requirements.txt for the triage service (lesson 14) and a Compose file with the lesson 15 networks: customer (internal: true) for the triage service and the customer API, and outside plus customer for the egress proxy; publish no ports to your laptop.

3. Set the Squid allowlist to one host, api.model-vendor.example (the dstdomain line of lesson 15's squid.conf), then send three requests from inside the triage container: the model host through the proxy (expect 200), example.org through the proxy (expect 403 from Squid) and a direct request with trust_env=False (expect httpx.ConnectError).

4. Run docker compose down and then docker compose up -d --build --wait five times, time each cold start with a shell timer, and record the median in seconds in README.md.

5. Build release 1.3.0 from the lesson 15 code with ERP_READER_CLIENT_SECRET renamed to ERP_READER_SECRET in the code only, so the Compose file still passes the old name; start it with TRIAGE_TAG=1.3.0 and up -d --build --wait (expect a KeyError at start), then roll back with TRIAGE_TAG=1.2.0 and up -d --wait. Time the outage from the bad up command to the first 200 on /triage/BB-1000, and check that the reader token gets 403 on a credits write and 401 on a token signed with a guessed key.

6. Run the lesson 16 slow wrapper with ERP_SLOW_MS=1500, send 8 requests to /triage/BB-1000, find the slow requests in the JSON logs (duration_ms of 1000 or more), and open one of their traces in Jaeger to name the slowest span and its duration in milliseconds.

7. Write three runbooks (triage slow, customer API rejects the token, bad release writes wrong labels), each with the five sections of lesson 17 (Symptom, First check, Mitigate, Verify, Escalate to), and save them as runbooks/*.md.

8. Copy the decisions SQLite file with the lesson 17 restore drill (sqlite3 Connection.backup into a scratch file), run its checks on the copy, and record the measured RTO in minutes against the 15-minute lab target and the RPO in hours against the 24-hour lab target.

Deliverable: A repo with the Dockerfile, the two Compose files (lesson 14 and lesson 15 versions), the Squid allowlist, the three runbooks and a test script that runs every check. The README has a controls table with the measured status codes for the three proxy checks, the scope and forged-token results, the median cold start in seconds, the bad-release outage and rollback time in seconds, the slowest span's name and duration in milliseconds, and the restore drill's RTO and RPO.

Stretch: Rotate ERP_READER_CLIENT_SECRET while the service runs, count the requests that fail during the rotation, and write how long the failure window lasts in seconds. · Add a Prometheus alert on the lesson 16 latency histogram that fires when p95 triage latency is over 2 seconds for 5 minutes, and show it firing on the slow run from the sixth step.

Project 3: A tool-using triage agent with a bounded tool surface and a cost budget

Goal: Give a Basket Bazaar triage agent three tools over an MCP server, keep the dangerous one away from the model, bound its steps and tokens, and measure cost and refusals across 25 runs, the way a team decides whether an agent is safe to put in front of customers.

Uses lessons: 05, 06, 11, 18, 19

1. Write an MCP server file with the mcp 2.x SDK (MCPServer, as in lesson 19) that exposes lookup_order, propose_credit (order_id, item, qty) and issue_refund; issue_refund stays on the server as the dangerous tool the model must never be offered.

2. Write a client that lists the server's tools with the MCP Client and offers the model only the allow-list (lookup_order and propose_credit); print the offered tool names and check that the count is 2.

3. Add server-side and agent-side checks: the order must exist in the lesson 05 orders file, a credit must name an item on that order, and a propose_credit above 2 units gets the refusal 'needs a person' (the within_policy rule of lesson 19); send 6 bad requests (unknown order, item not on the order, qty 3, qty 0, an issue_refund request and an unknown tool name) and check each is refused with no model call.

4. Cap the agent at MAX_STEPS = 3 model calls per customer message and TOKEN_BUDGET = 1500 input tokens per call (lesson 18), trim the history to CONTEXT_BUDGET = 400 tokens with trim_messages, and stop the run with a clear message when a limit is hit.

5. Run 25 invented customer messages (5 per issue type, seed 19) through the agent with the lesson 18 BudgetedLRU answer cache set to max_entries=50 and ttl_s=3600 on order lookups, and write a run log with one JSON line per run: model calls, input tokens, latency in milliseconds, cache hits and refused tool calls.

6. Compute from the 25 runs the share that finish in 3 or fewer model calls, the median and p95 latency in milliseconds, the cache hit rate, the refused-call count by reason, and the cost per resolved case at the lesson 18 prices (USD 0.25 per million input and 1.50 per million output, dated 2026-10-22); a resolved case is a run that ends with an answer and no 'needs a person' refusal.

7. Write tests/test_guardrails.py with pytest that replays the run log and fails if issue_refund is ever offered or called, if any propose_credit above 2 units reaches the server, or if any run uses more than 3 model calls.

Deliverable: A repo with the MCP server, the client, the 25-message file, the run log (one JSON line per run), tests/test_guardrails.py and README.md. The README reports the measured numbers from your run: runs finishing in 3 or fewer calls, median and p95 latency in milliseconds, cache hit rate, refused calls by reason, cost per resolved case with the price date, and the pytest result, plus one paragraph on what the guardrails do not stop (for example a wrong but allowed credit, or a reply that promises something the team cannot give).

Stretch: Store each propose_credit call in a SQLite table with a status column, add a second script that approves or rejects them, and measure how many proposals wait longer than 1 hour in a simulated day of 200 messages. · Compare the cost of the same 25 runs with and without the cache and state the saving in percent against a monthly budget you set in lesson 18.

Project 4: City 311 service-request desk: triage 600 requests and ship it into a private network

Goal: Apply the same loop to a different domain, a city council's 311-style service-request desk with invented data: clean a messy export, triage with structured output, gate the release on measured per-category accuracy, and deploy it on a private network, the job a forward deployed engineer does at a public-sector or utility customer.

Uses lessons: 05, 06, 10, 11, 12, 14, 15, 16, 18

1. Generate 600 invented service requests with seed 311 into data/requests.csv with the columns request_id, ward (1 to 8, blank for about 2 percent), submitted_at and text: six kinds of problem (pothole, missed bin collection, streetlight out, noise, graffiti, other), typos and mixed case, about 5 percent exact duplicate rows, and no real names or addresses.

2. Remove the duplicates on (request_id, ward, text), write data/clean.csv with blank wards set to UNKNOWN, and record the row counts before and after and the number of rows dropped in README.md.

3. Write a triage function that returns a Pydantic model with category (a Literal of the six values) and urgency (an int from 1 to 3), then hand-label 120 of the cleaned rows (20 per category, chosen with seed 311) before the first model call and save them as data/eval_120.csv.

4. Run the triage on the 120 rows and print per-category precision and recall with sklearn.metrics.classification_report, then apply one code rule: a row whose text contains one of six safety words you write (for example wire, gas, fallen, injur, fire, sinkhole) must get urgency 3, and count the downgrades.

5. Write tests/test_release_gate.py with pytest: it fails when macro-F1 (sklearn f1_score with average='macro') is below 0.85 or when any safety row has urgency below 3.

6. Deploy the service with Compose on an internal network with an egress allowlist for the model host only (the lesson 15 pattern), then process the 600 requests in 12 batches of 50 and record the requests per minute and the input and output tokens per batch.

7. Add an OpenTelemetry span per batch (lesson 16) and a prometheus_client Counter labelled by category, find the slowest batch in Jaeger, and write its duration in seconds in the README.

Deliverable: A repo with the generator (seeded), the cleaned and labelled data, the triage service, the Compose files, tests/test_release_gate.py and README.md. The README reports from your run: duplicates dropped, the per-category precision and recall table, macro-F1 against the 0.85 gate, the number of urgency-3 downgrades (the target is 0), requests per minute, cost per 1,000 requests in USD at the lesson 18 prices with the price date, and the slowest batch's duration.

Stretch: Add 60 Spanish-language requests to the generator, report the macro-F1 gap against English, and decide whether the gate should be per language. · Route every urgency-3 request to a human queue table and measure the share of the day's traffic that goes there over 600 requests.